

User Manual

If you need examples of the operations or data types mentioned in the following sections, check the provided `examples.txt` file or refer to the end of each section for more information.

Compile and run the interpreter

```
make
make run # This will launch the interpreter with "ledit" and the parameter OCAMLRUNPARAM=b
         in order to be able to show the stacktrace in case an unhandled exception is
         raised
```

0. Basic Operations

First, note that **variable identifiers** must **start** with a **lowercase letter**, followed by an alphanumeric combination of **numbers** and **lowercase letters**, and may also include the character “_”.

Similarly, the names of **types** must **start** with an **uppercase letter**, excluding the character “L”, which is reserved for **Lambda** (*short version*).

We support the following basic data types and operations:

```
true (Bool)
false (Bool)
0~131037 (Nat) # Beyond this range, a stack overflow will occur (at least with the default
               OCaml 4.14.1 stack size)
succ x # Where x is a number within the range above or the output of a function
pred x
iszero x
if TERM1 then TERM2 else TERM3
lambda x : TYPE_OF_X . TERM # To declare a function
L x : TYPE_OF_X . TERM
FUNCTION TERM # Application of a declared function to a term
LET ID = TERM1 in TERM2 #
```

- Examples:

```
0
succ (succ (succ 0))
3
succ (pred 0)
iszero (pred (succ (succ 0)))
if iszero 3 then 0 else 1
iszero true (*type error: argument of iszero is not a number*)
if 1 then true else false (*type error: guard of conditional not a boolean*)
if iszero 3 then 0 else false (*type error: arms of conditional have different
                              types*)
let id = Lx.x in id 3 (*syntax error*)
let id_bool = L x:Bool. x in id_bool true
let id_nat = L x:Nat. x in id_nat 5
let x = 5 in let id_nat = L x:Nat. x in id_nat x
(*Syntax error in both for not being typed*)
let fix = lambda f.(lambda x. f (lambda y. x x y)) (lambda x. f (lambda y. x x y)) in let
  sumaux = lambda f. (lambda n. (lambda m. if (iszero n) then m else succ (f (pred
  n) m))) in let sum = fix sumaux in sum 21 34
let fix = lambda f.(lambda x. f (lambda y. x x y)) (lambda x. f (lambda y. x x y)) in let
  sumaux = lambda f. (lambda n. (lambda m. if (iszero n) then m else succ (f (pred
  n) m))) in let sum = fix sumaux in let prodaux = lambda f. (lambda n. (lambda m.
  if (iszero m) then 0 else sum n (f n (pred m)))) in let prod = fix prodaux in
  prod 12 5
```

1. Multiline Expressions

The user can input multiline expressions like the following:

```
>> lambda x : Nat .  
    succ (succ x)  
;;
```

This will produce a result similar to:

```
(lambda x:Nat. (succ (succ x))) : Nat -> Nat
```

Lines will be read until the EOF token is found, which is “;;”. Anything written after this token will be ignored.

2. Pretty-printer

The interpreter output has been made more user-friendly when entering lambda expressions.

For example, if the user types the following:

```
>> letrec sum:Nat->Nat->Nat =  
    Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)  
in letrec mul:Nat->Nat->Nat =  
    Ln:Nat.Lm:Nat. if iszero n then 0 else sum m (mul (pred n) m)  
;;
```

The output will be:

```
letrec sum:Nat -> (Nat -> Nat) =  
    (lambda n:Nat. (lambda m:Nat. if (iszero n) then m else (succ ((sum (pred n))  
m))))  
in letrec mul:Nat -> (Nat -> Nat) =  
    (lambda n:Nat. (lambda m:Nat. if (iszero n) then 0 else (((sum m) ((mul (pred n))  
m))))),  
in mul : Nat -> (Nat -> Nat)
```

In this example, which will be further explained in the *Recursion* section, it can be observed that **mul** is a function of:

```
Nat -> Nat -> Nat
```

However, in this case, it appears as:

```
Nat -> (Nat -> Nat)
```

This is due to the internal structure of the code, but it can be interpreted as a function that, given the first **Nat**, returns a function **Nat -> Nat** which, when applied to the second **Nat**, returns a **Nat**.

3. Recursion

As shown in the previous section, a recursive function declaration follows this structure:

```
letrec FUNCTION_ID : FUNCTION_TYPE = TERM_FOR_RECURSIVE_EVALUATION [in  
    TERM_THAT_USES_THE_FUNCTION]
```

For example, in the previous **mul** example, the internal fixed-point operator is not exposed. In fact, if we now call the **mul** function from the context, we will get something like this:

```
>> mul;;
```

```
mul : Nat -> (Nat -> Nat)
```

However, the interpreter also supports expressions with “**fix**”, although their consistency and correctness are the user’s responsibility.

- Examples:

```
letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in sum 23 27;;

letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in letrec mul:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then 0 else sum m (mul (pred n) m) in mul 20 40;;

letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in letrec fib:Nat->Nat =
  Ln:Nat. if iszero n then 0 else if iszero (pred n) then 1 else sum (fib (pred n)) (fib
    (pred (pred n)))
in fib 15;; (*Previously, we could compute up to fib 16, but adding the eval conditional
  fills the stack sooner*)

let sub = letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in letrec sub:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero m then n else sub (pred n) (pred m)
;;
```

4. Global Context

Regarding the global context, we have modified the **let x = ... in ...** and **letrec x : ... = ... in ...** expressions shown earlier so that if “**in**” is not specified, these will be saved in the context.

It is important to note that if you have chained **let** or **letrec** declarations, they will not be saved automatically, and you will need to save them manually. For example, this will first be saved as **sum**:

```
letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
;;
```

However, this will not be saved automatically. You must specify **how** you want it saved at the beginning (*by adding **let mul =** at the start of the expression*):

```
let mul = letrec sum:Nat -> (Nat -> Nat) =
  (lambda n:Nat. (lambda m:Nat. if (iszero n) then m else (succ ((sum (pred n))
    m))))
in letrec mul:Nat -> (Nat -> Nat) =
  (lambda n:Nat. (lambda m:Nat. if (iszero n) then 0 else (((sum m) ((mul (pred n))
    m)))))
```

The same applies to a normal **let**:

```
let x = 5;; # This will save it as x

let y = let x = 5 in let y = succ x;; # You need to write let y = at the beginning;
  otherwise, it will simply return 6 but won't assign it to "y"
```

As for the type alias, you have to follow this format

```
type TYIDV = TYPE
```

- Examples:

```
(* Functional global context *)
let x = 3;;
let id = Lz:Nat.x;; (* (lambda z:Nat. 3) : Nat -> Nat *)
(* Even if x is later modified, the id function will still return 3 *)

(* FUNCTIONS *)
(* SUM *)
letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
;;

(* MUL *)
let mul = letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in letrec mul:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then 0 else sum m (mul (pred n) m)
;;

(* FIB *)
let fib = letrec sum:Nat->Nat->Nat =
  Ln:Nat.Lm:Nat. if iszero n then m else succ (sum (pred n) m)
in letrec fib:Nat->Nat =
  Ln:Nat. if iszero n then 0 else if iszero (pred n) then 1 else sum (fib (pred n)) (fib
    (pred (pred n)))
;;

(* ALIAS *)
type N3 = Nat->Nat->Nat;; (* N3 : Nat -> (Nat -> Nat) *)
(Lx:N3.Ln:Nat.Lm:Nat. x n m) sum 5 6;; (* 11 : Nat *)
```

5. String Type

To declare a string, simply put the text you want to use inside " " (*Inside the quotes, all characters are allowed **except for** " ; , the quote character itself, and newlines*).

To concatenate two strings, the **concat** function is used.

- Examples:

```
concat "Hello " (concat "I am " (concat "a " "String"));; (* "Hello I am a String" :
String *)
```

6. Tuples

Tuples are a sequence of terms of any type, separated by commas, and enclosed in ().

To access a specific position in the tuple (projection), use the “#” operator.

- Examples:

```
(0,1,2,3,4,5) #3;; (* 3 : Nat *)
(0,1,2,3,4,5) #200;; (* Projection index out of bounds *)
(1,true,"hello",24,Lx:Nat.x);; (* (1,true,"hello",24,(lambda x:Nat. x)) :
(Nat,Bool,String,Nat,Nat -> Nat) *)
let x = (1,(2,3),4);; (* (1,(2,3),4) : (Nat,(Nat,Nat),Nat) *)
x #1 #1;; (* 3 : Nat *)
```

7. Records

For records, we followed a similar approach to OCaml, where you first need to define the record type, and later instances of that type can be created.

- Examples:

```
type Person = {name:String; age:Nat};;
let x = Person {name = "John"; age = 32};; (*Person {name="John";age=32} : Person =
{name:String;age:Nat}*)
x.name;; (*"John" : String*)
type Almendra = {name:String; age:Nat; palomita:Person};;
let p = Person {name="Matengo"; age=21};;
let almond = Almendra {name="Matengando"; age=999; palomita=p};;
type Algo = {persona:Person; almendra:Almendra};;
let something = Algo {persona=p; almendra=almond};;
something.almendra.palomita.name;; (*"Matengo" : String*)
```

8. Variants

In variants, you first need to define the type, and then you can create instances using **<LABEL=VALUE> as TYPE**.

- Examples:

```
type Int = <pos:Nat; zero:Bool; neg:Nat>;;
let p3 = <pos=3> as Int;;
let n4 = <neg=4> as Int;;
let abs = Lx:Int.
  case x of
  <pos=p> -> <pos=p> as Int
  | <zero=z> -> <zero=z> as Int
  | <neg=n> -> <pos=n> as Int
;;
abs p3;; (*<pos=3> as Int : Int = <pos:Nat;zero:Bool;neg:Nat>*)
abs n4;; (*<pos=4> as Int : Int = <pos:Nat;zero:Bool;neg:Nat>*)

(*define sum and sub before this function*)
let add =
  Lx:Int.Ly:Int.
  case x of
  <pos=p1> ->
    (case y of
    <pos=p2> -> <pos=(sum p1 p2)> as Int
    | <zero=z2> -> <pos=p1> as Int
    | <neg=n2> ->
      if iszero (sub n2 p1) then <pos=(sub p1 n2)> as Int
      else (if iszero (sub p1 n2) then <neg=(sub n2 p1)> as Int
      else <zero=true> as Int))
  | <zero=z1> ->
    (case y of
    <pos=p2> -> <pos=p2> as Int
    | <zero=z2> -> <zero=true> as Int
    | <neg=n2> -> <neg=n2> as Int)
  | <neg=n1> ->
    (case y of
    <pos=p2> ->
      if iszero (sub p2 n1) then <neg=(sub n1 p2)> as Int
      else (if iszero (sub n1 p2) then <pos=(sub p2 n1)> as Int
      else <zero=true> as Int)
    | <zero=z2> -> <neg=n1> as Int
    | <neg=n2> -> <neg=(sum n1 n2)> as Int)
  ;;
add p3 n4;; (*<neg=1> as Int : Int = <pos:Nat;zero:Bool;neg:Nat>*)
```

```
add n4 p3;; (*<neg=1> as Int : Int = <pos:Nat;zero:Bool;neg:Nat>*)
```

9. Lists

Lists, like tuples, are a sequence of terms separated by commas, but enclosed in “[]”, with the peculiarity that **all** elements must be of the same type.

For operations on lists, we have **hd**, **tl**, **isempty**, and the operator “::” (*concatenation*).

The **concatenation** expression should be enclosed in **parentheses**, for example:

```
(1::[2,3,4,5])
```

- Examples:

```
[1,2,"hello",3];; (*type error: Type mismatch: All elements of the list should be the same
                    type*)
[1,2,3,4];; (*[1,2,3,4] : [Nat]*)
isempty [1,2];; (*false : Bool*)
hd [1,2];; (*1 : Nat*)
tl [1,2,3];; (*[2,3] : [Nat]*)
(1::[2,3,4,5]);; (*[1,2,3,4,5] : [Nat]*)
(1::(hd tl hd tl [[1],[3]],[2],[4]])) (*[1,4] : [Nat]*)

(*Top is for generic lists (of any type), since it is the supertype of any type. If you
   want, for example, to apply the function to Nat lists, simply change it to
   [Nat]*)
letrec length:Top->Nat =
  Li:Top. if isempty i then 0 else succ (length (tl i))
;;
letrec append:Top->Top->Top =
  L1:Top.L2:Top.
    if isempty l1 then l2 else (hd l1 :: (append (tl l1) l2))
;;
(* (T->U) -> [T] -> [U] *)
letrec map:Top->Top->Top =
  Lf:Top->Top.Llst:Top.
    if isempty lst then []
    else ((f (hd lst)) :: (map f (tl lst)))
;;

length [1,2,3,4];; (*4 : Nat*)

append [1,2,3,4] [5,6,7,8];; (*[1,2,3,4,5,6,7,8] : Top*)

map fib [1,2,3,4,5,6,7,8];; (*[1,1,2,3,5,8,13,21] : Top*)
```

10. Subtyping

For subtyping, only the following types are considered:

```
S <: S
S <: U && U <: T => S <: T
T1 <: S1 && S2 <: T2 => S1->S2 <: T1->T2
S <: Top
Bot <: T
# Subtyping for records
{li:Ti}i=1..n+k <: {li:Ti}i=1..n
Vi Si <: Ti => {li:Si}i=1..n <: {li:Ti}i=1..n
# The order of the fields in a record does not matter for subtyping
```

- Examples:

```

type Person = {name:String; age:Nat};;
let x = Person {name = "John"; age = 32};; (*Person {name="John";age=32} : Person =
      {name:String;age:Nat}*)
x.name;; (*"John" : String*)

(*Subtyping*)
type Person2 = {name:String; age:Nat; palomita:Bool};;
let f = Lp:Person.p.name;; (*(lambda p:Person. p.name) : (Person = {name:String;age:Nat})
      -> String*)
let p2 = Person2 {name = "John"; age = 32; palomita = false};;
f p2;; (*"John" : String*)

let f2 = Lp:{}.p;; (*(lambda p:empty_record = {}. p) : (empty_record = {}) ->
      (empty_record = {})*
f2 p2;; (*Person2 {name="John";age=32;palomita=false} : empty_record = {}*)

(*Identity function 'a -> 'a*)
Lx:Top.x;; (*(lambda x:Top. x) : (Top) -> (Top)*)

```